Instrumentos Computacionales - Stata

Maestría en Economía

Santiago Cerutti

UNLP

Marzo 2026

Organización del curso

Estructura

- 3 clases de Stata (docente: Santiago Cerutti).
- 3 clases de R (docente: Cristian Bonavida).

Evaluación

- Cada parte (Stata y R) tiene examen y trabajos prácticos.
- Ponderación en cada parte: examen 80 % y TP 20 %.

Aprender a programar en la era de la IA

- Hoy la IA puede escribir código, automatizar tareas y acelerar el análisis.
- Esto cambia la forma en que trabajamos con datos.

Aprender a programar en la era de la IA

- Hoy la IA puede escribir código, automatizar tareas y acelerar el análisis.
- Esto cambia la forma en que trabajamos con datos.

¿Tiene sentido aprender a programar?

Aprender a programar en la era de la IA

1. La IA ejecuta, pero no valida el análisis económico:
   - No evalúa si los resultados tienen sentido económico.
   - No conoce cómo se generaron los datos.
   - Los outputs dependen del prompt y del modelo que usemos

Aprender a programar en la era de la IA

1. La IA ejecuta, pero no valida el análisis económico:
   - No evalúa si los resultados tienen sentido económico.
   - No conoce cómo se generaron los datos.
   - Los outputs dependen del prompt y del modelo que usemos

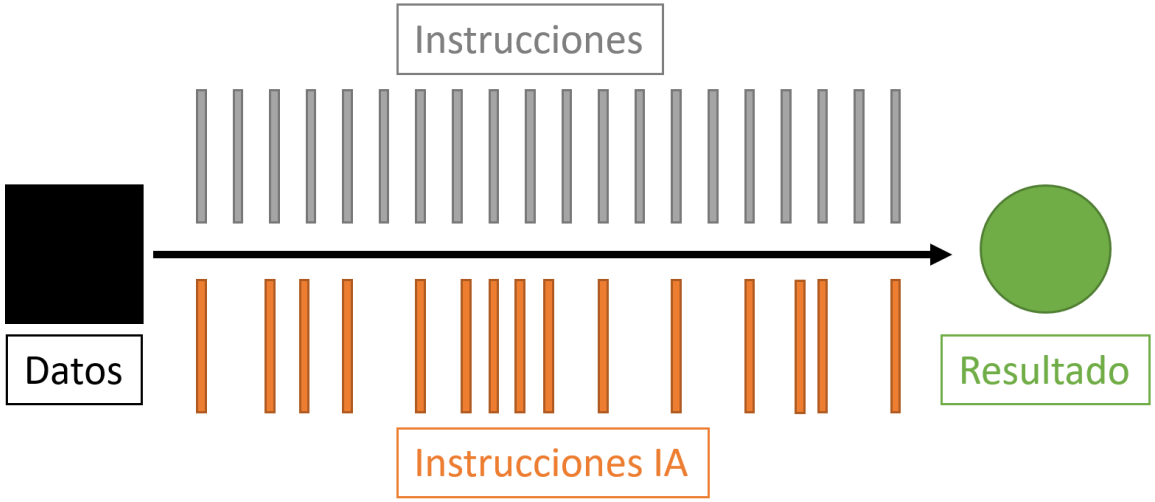
2. Saber programar te da control sobre el proceso:
   - Detectar errores
   - Revisar resultados
   - Adaptar el código

La IA **reduce el costo de escribir código**,
pero **aumenta el valor de entenderlo.**

Aprender a programar en la era de la IA

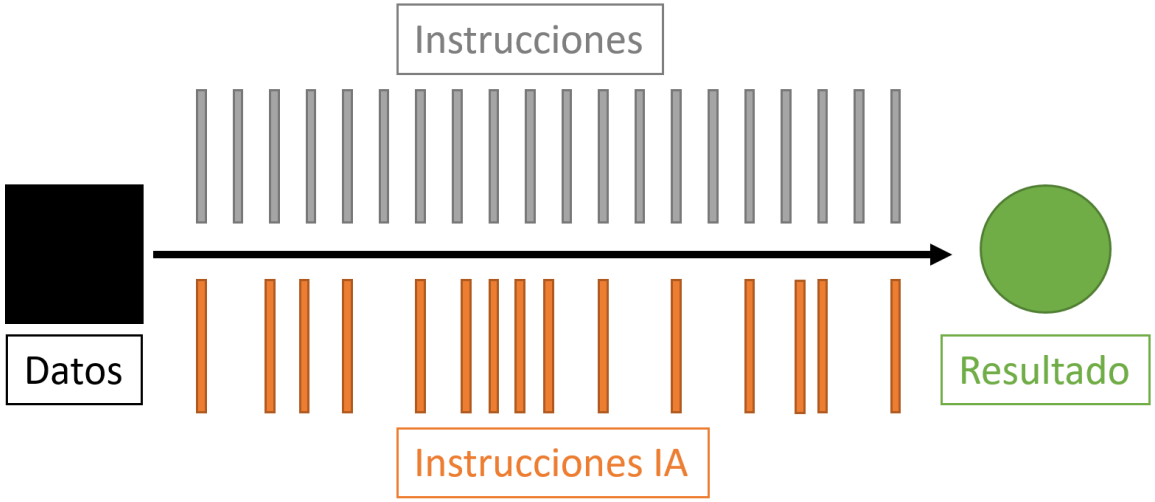
- La IA no nos reemplaza, nos **potencia**
- La clave hoy no está en **programar** vs **no programar**
- Sino en **usar IA con criterio** vs **depender de ella**

Aprender a programar en la era de la IA



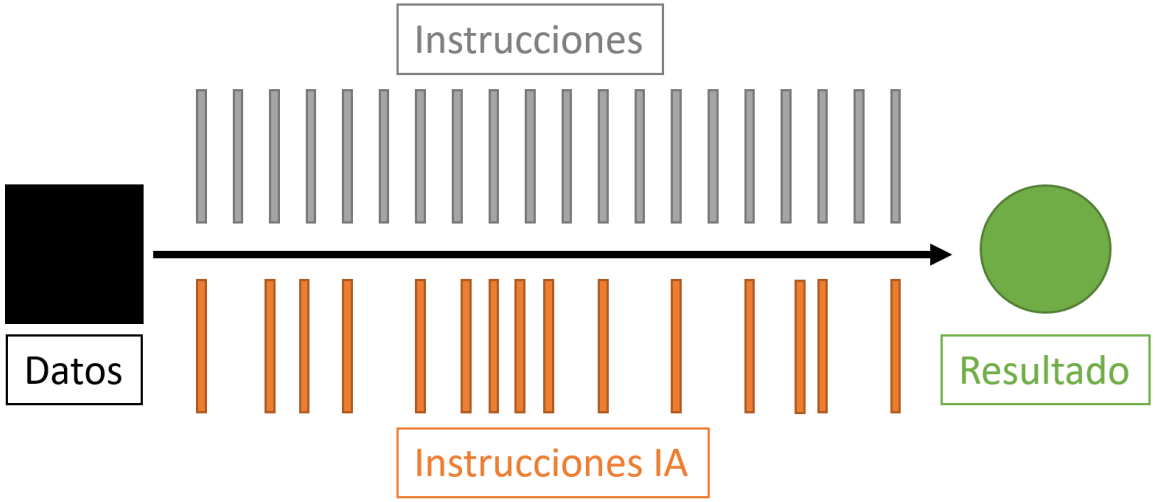
- **1) Resultado:** la IA puede llegar a una respuesta correcta, pero por un camino distinto.

Aprender a programar en la era de la IA



- **1) Resultado:** la IA puede llegar a una respuesta correcta, pero por un camino distinto.
- **2) Validacion:** si no entendemos ese camino, no podemos verificar ni explicar el resultado.

Aprender a programar en la era de la IA



- **1) Resultado:** la IA puede llegar a una respuesta correcta, pero por un camino distinto.
- **2) Validacion:** si no entendemos ese camino, no podemos verificar ni explicar el resultado.
- **3) Criterio:** para construir instrucciones, necesitamos saber un lenguaje.

¿Por qué Stata?

- En economía aplicada, el manejo de datos y la estimación econométrica son centrales.
- Es verdad que existe un **cambio tecnológico** en curso:
  - R y Python crecen rápidamente (sobretodo para big data y machine learning)

¿Por qué Stata?

- En economía aplicada, el manejo de datos y la estimación econométrica son centrales.
- Es verdad que existe un **cambio tecnológico** en curso:
  - R y Python crecen rápidamente (sobretodo para big data y machine learning)
- Pero la profesión mantiene una **fuerte inercia**.
- **Stata sigue siendo el lenguaje dominante**:
  - en papers empíricos
  - en organismos internacionales y policy

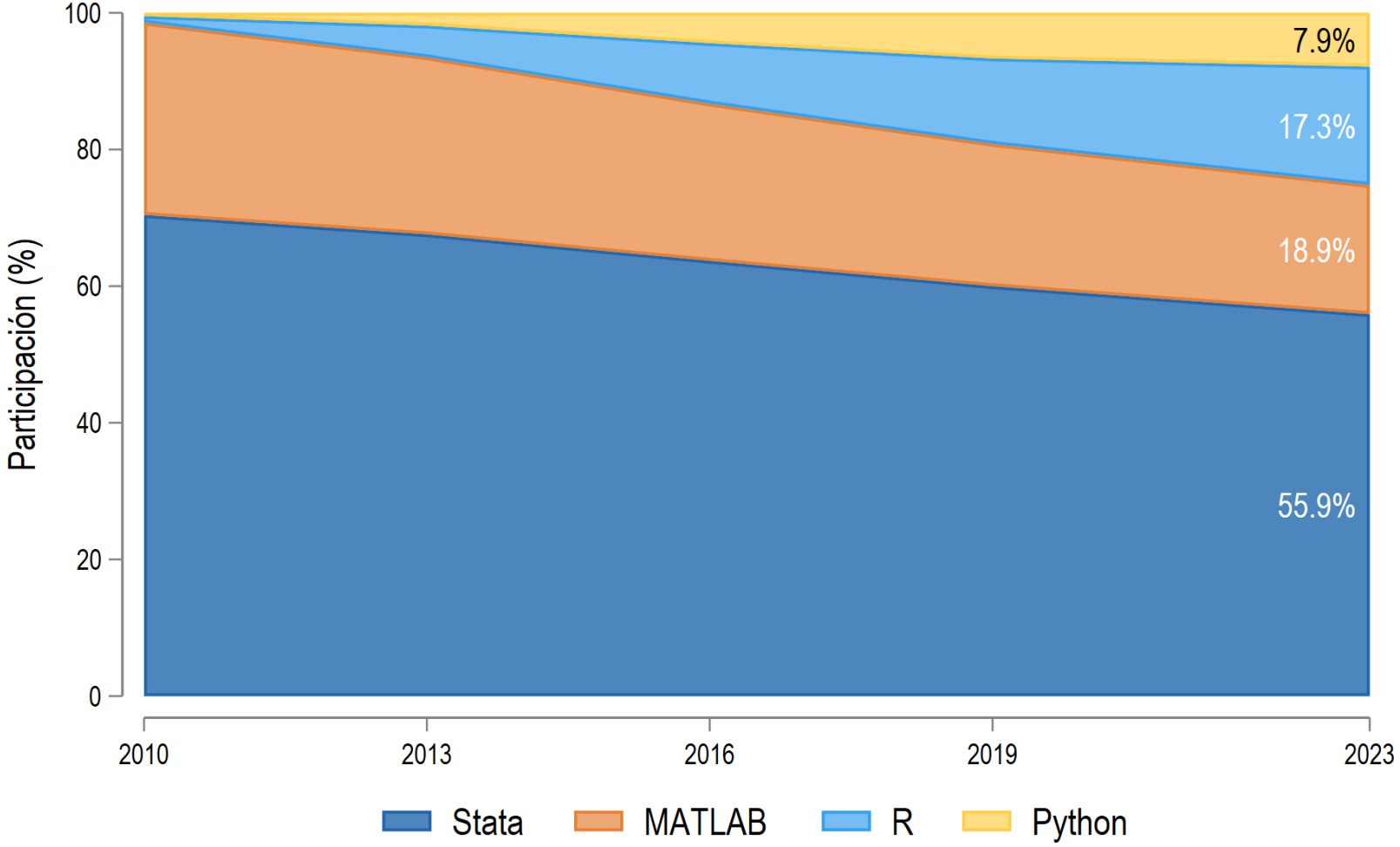
¿Por qué Stata?

- En economía aplicada, el manejo de datos y la estimación econométrica son centrales.
- Es verdad que existe un **cambio tecnológico** en curso:
  - R y Python crecen rápidamente (sobretodo para big data y machine learning)
- Pero la profesión mantiene una **fuerte inercia**.
- **Stata sigue siendo el lenguaje dominante**:
  - en papers empíricos
  - en organismos internacionales y policy

No saber Stata hoy sigue siendo una desventaja laboral importante.

¿Por qué Stata?

Figura 1: Uso de lenguajes de programación en economía (academia)



Fuente: elaboración propia en base a R-bloggers.

Objetivos del curso

- Comprender la sintaxis general del lenguaje.
- Saber limpiar, transformar, organizar y analizar bases de datos.
- Resolver problemas comunes con enfoque práctico.
- Promover el uso de código ordenado y reproducible.

Programa

- Clase 1:
  - Recomendaciones y nociones básicas de programación.
  - Interfaz y sintaxis general.
  - Gestión básica de base de datos.
  - Estadísticas descriptivas y gráficos básicos.
  - Do-files.
- Clase 2:
  - Macros y loops.
  - Funciones.
  - Gestión avanzada de base de datos.
- Clase 3:
  - Organización de proyectos.
  - Múltiples do-files.

Contenido Clase 1

1. Recomendaciones
2. Interfaz y sintaxis general
3. Gestión básica de base de datos
4. Estadísticas descriptivas
5. Gráficos
6. Do-files
7. Trabajo Práctico 1

Contenido Clase 1

1. Recomendaciones

2. Interfaz y sintaxis general

3. Gestión básica de base de datos

4. Estadísticas descriptivas

5. Gráficos

6. Do-files

7. Trabajo Práctico 1

Editores de texto

- El editor de texto es una herramienta que sirve para listar las instrucciones que Stata debe ejecutar.
- En qué software escribimos ese código es indistinto.
- Editores recomendados:
  - Visual Studio Code (VS Code)
  - Editor de texto integrado en Stata
  - Notepad++

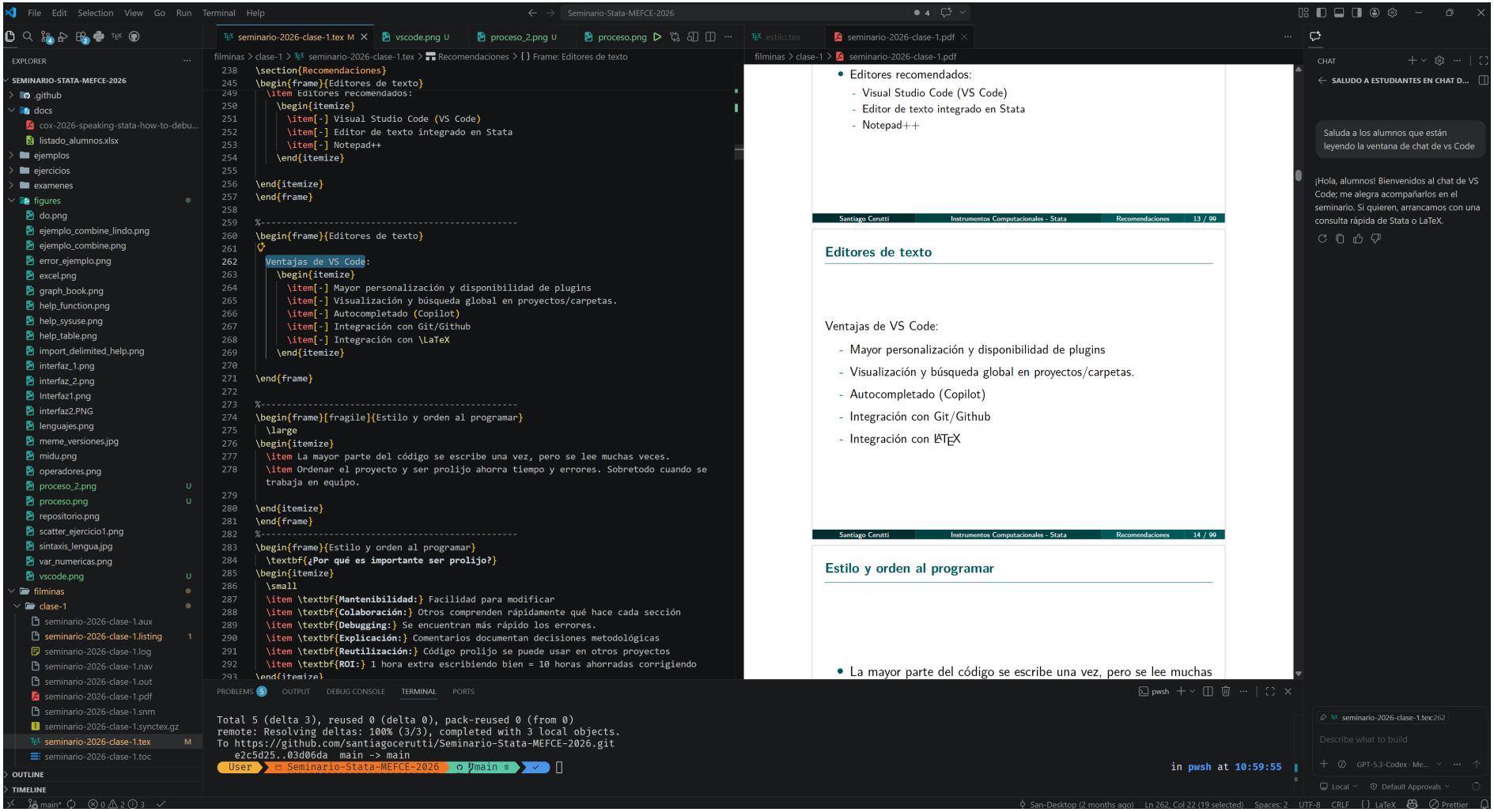
Editores de texto

Ventajas de VS Code:

- Mayor personalización y disponibilidad de plugins
- Visualización y búsqueda global en proyectos/carpetas.
- Autocompletado (Copilot) y Agentes de IA.
- Integración con Git/Github
- Integración con LaTeX

Editores de texto

VS Code:



Estilo y orden al programar

- La mayor parte del código se escribe una vez, pero se lee muchas veces.
- Ordenar el proyecto y ser prolijo ahorra tiempo y errores. Sobretodo cuando se trabaja en equipo.

Estilo y orden al programar

¿Por qué es importante ser prolijo?

- **Mantenibilidad:** Facilidad para modificar
- **Colaboración:** Otros comprenden rápidamente qué hace cada sección
- **Debugging:** Se encuentran más rápido los errores.
- **Explicación:** Comentarios documentan decisiones metodológicas
- **Reutilización:** Código prolijo se puede usar en otros proyectos
- **ROI:** 1 hora extra escribiendo bien = 10 horas ahorradas corrigiendo

Estilo y orden al programar

Ejemplo:

```
use "datos.dta" , clear
gen x= .
replace x = 1 if y == 1
replace x=2 if y==2
replace x =3 if y==3
sum x
```

Comparado con:

```
* Cargar datos originales
use "datos.dta" ,  clear

* Crear variable
gen     x = .
replace x = 1 if y == 1
replace x = 2 if y == 2
replace x = 3 if y == 3

* Estadísticas descriptivas
summarize x
```

Cómo se aprende a programar

- A programar se aprende... programando.
- Ver clases y tutoriales ayuda, pero no es suficiente.
- Leer código tampoco es suficiente.
- Equivocarse es parte del proceso de aprendizaje.
- Saber leer e interpretar los errores es una habilidad clave.

Git y GitHub

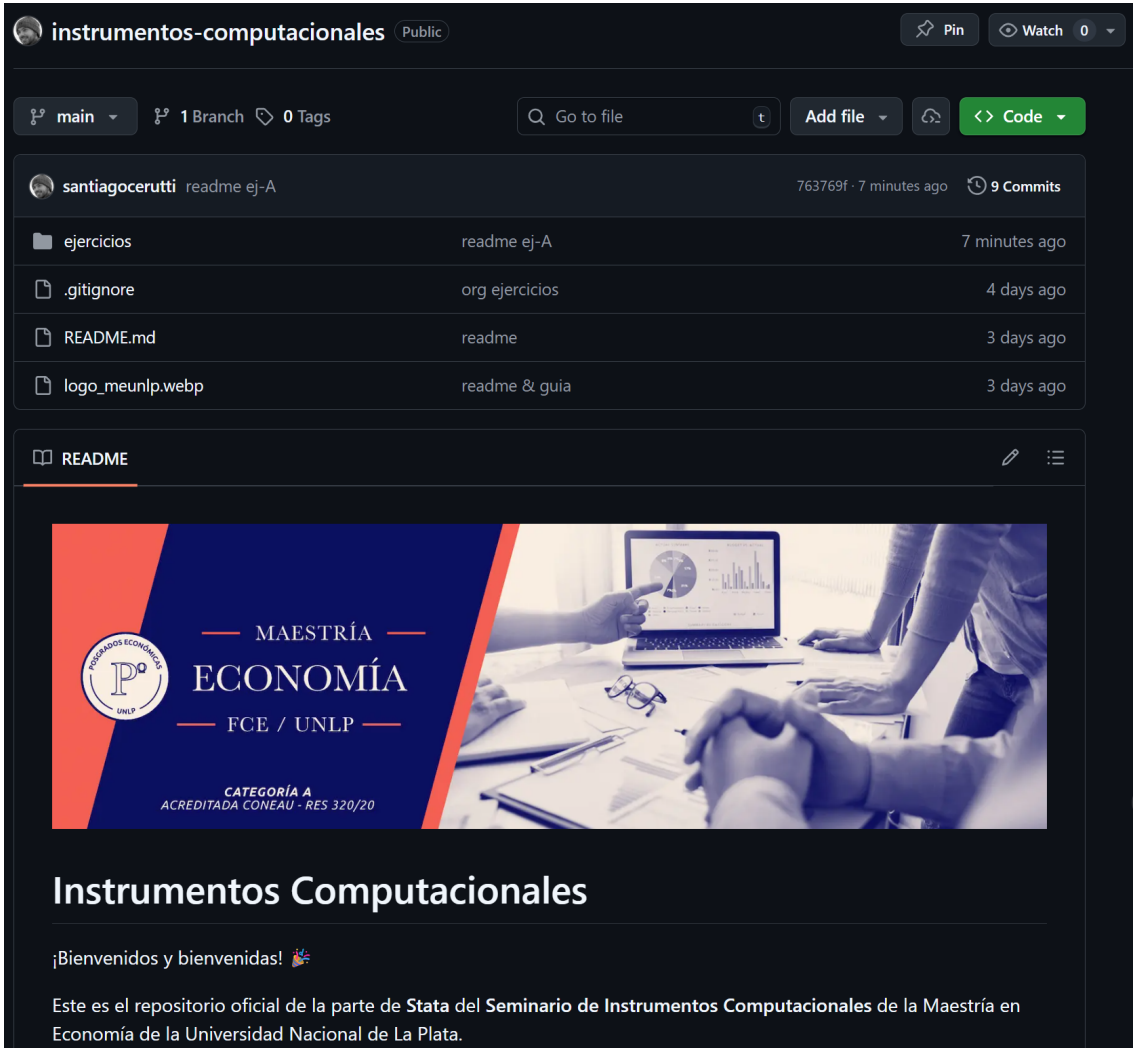
Para evitar:

- Super Cool Report v1.xlsx
- Super Cool Report v2.xlsx
- Super Cool Report v3.1.xlsx
- Super Cool Report v3.xlsx
- Super Cool Report v4.xlsx
- Super Cool Report v4a.xlsx
- Super Cool Report v4b.xlsx
- Super Cool Report v5.xlsx
- Super Cool Report vFinal.xlsx
- Super Cool Report vFinal_1.xlsx
- Super Cool Report vFinal_2.xlsx
- Super Cool Report vFinal_Final.xlsx
- Super Cool Report vFinal_Final-UPDATED.xlsx
- Super Cool Report vFinal_Final-UPDATED_NEW.xlsx

Git y GitHub

- Control de versiones para archivos de texto plano.
- Permite:
  - Volver atrás (a cualquier versión en cualquier momento)
  - Documentar cambios
  - Trabajar en equipo
- Su uso no es obligatorio para este curso pero es altamente recomendable aprender a usarlo.
- Tienen a disposición el repositorio del curso (`https://github.com/santiagocerutti/instrumentos-computacionales`), donde podrán encontrar el material de cada clase, ejercicios y otros recursos.

Git y GitHub



Repositorio del curso en GitHub

Uso correcto de IA

- Acelera procedimientos y puede ayudarnos en el aprendizaje pero no reemplaza el entendimiento de los conceptos.
- Puede generar errores sutiles y difíciles de detectar.
- Particularmente para Stata suele confundirse con Python.
- Regla: nunca ejecutar código que uno no entienda o pueda explicar.
- Sumamente útil para:
  - Autocompletado de código.
  - Entender errores.
  - Emprolijar y organizar código.
  - Documentar.
  - Agilizar procesos sencillos (y repetitivos).

Contenido Clase 1

1. Recomendaciones

2. Interfaz y sintaxis general

3. Gestión básica de base de datos

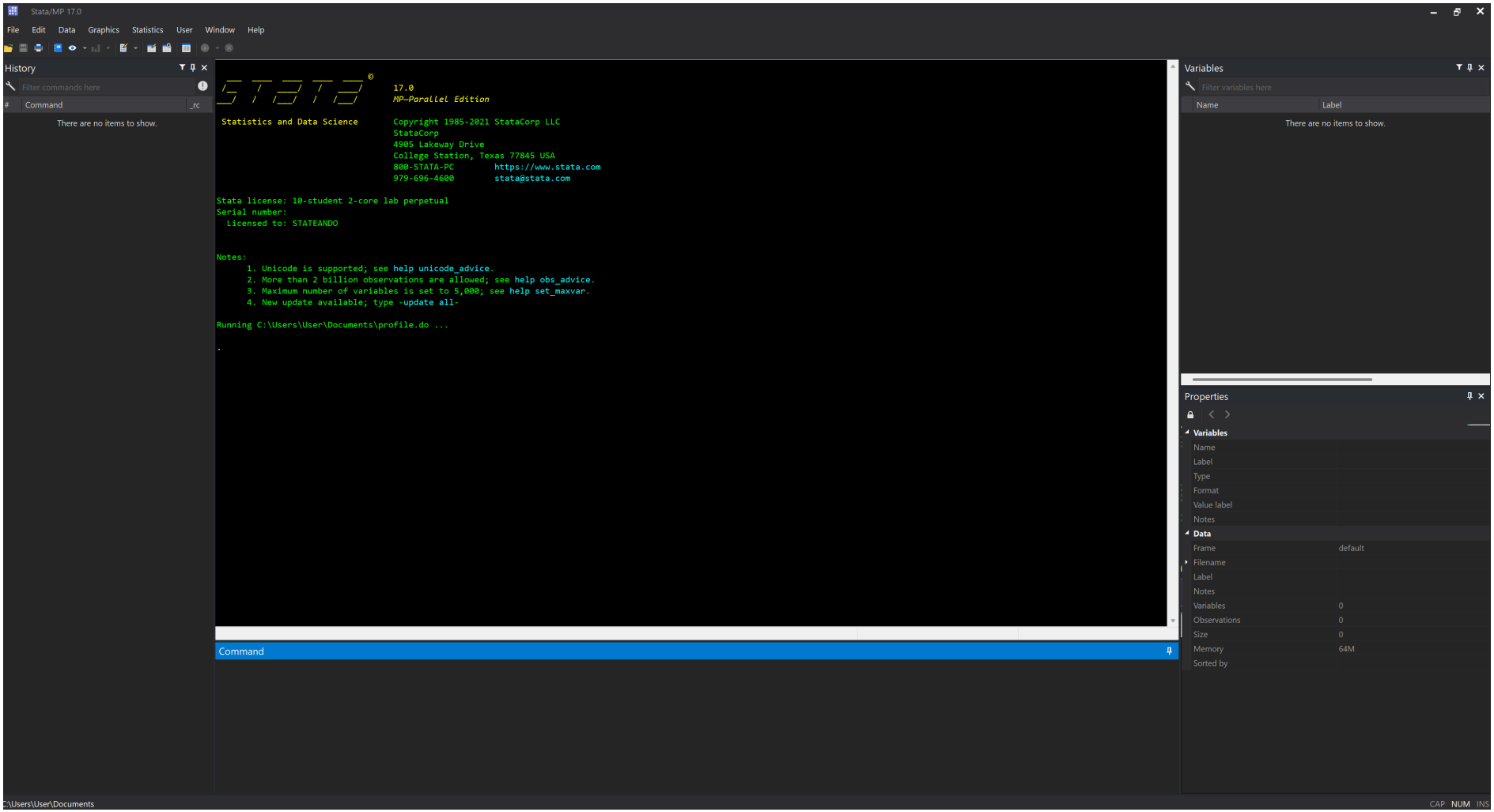
4. Estadísticas descriptivas

5. Gráficos

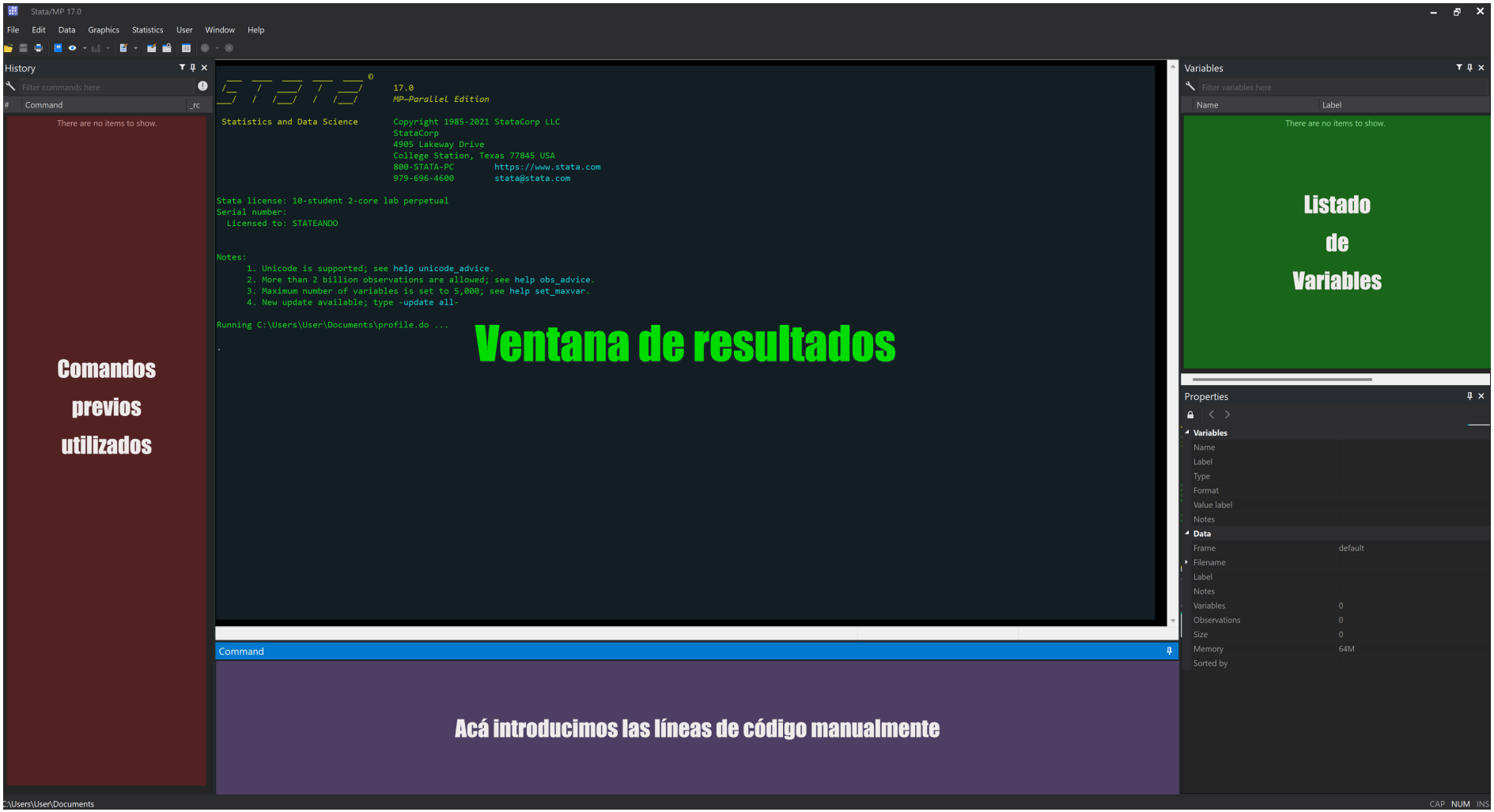
6. Do-files

7. Trabajo Práctico 1

Interfaz


Stata/MP 17.0
File Edit Data Graphics Statistics User Window Help
History
Filter commands here
Command _rc
There are no items to show.
17.0
MP-Parallel Edition
Statistics and Data Science
Copyright 1985-2021 StataCorp LLC
StataCorp
4905 Lakeway Drive
College Station, Texas 77845 USA
800-STATA-PC https://www.stata.com
979-696-4600 stata@stata.com
Stata license: 10-student 2-core lab perpetual
Serial number:
Licensed to: STATEANDO
Notes:
1. Unicode is supported; see help unicode_advice.
2. More than 2 billion observations are allowed; see help obs_advice.
3. Maximum number of variables is set to 5,000; see help set_maxvar.
4. New update available; type -update all-
Running C:\Users\User\Documents\profile.do ...
Command
Variables
Filter variables here
Name Label
There are no items to show.
Properties
Variables
Name
Label
Type
Format
Value label
Notes
Data
Frame default
Filename
Label
Notes
Variables 0
Observations 0
Size 0
Memory 64M
Sorted by
C:\Users\User\Documents
CAP NUM INS

Interfaz

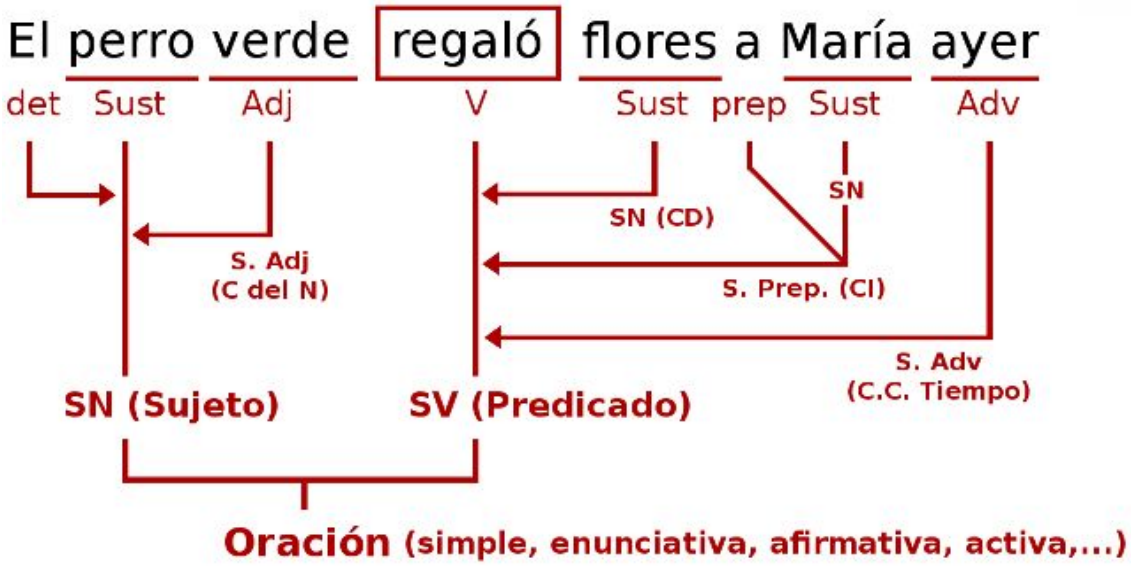

Stata/MP 17.0
File Edit Data Graphics Statistics User Window Help
History
Filter commands here
Command _rc
There are no items to show.
Comandos previos utilizados
17.0
MP-Parallel Edition
Statistics and Data Science
Copyright 1985-2021 StataCorp LLC
StataCorp
4905 Lakeway Drive
College Station, Texas 77845 USA
800-STATA-PC https://www.stata.com
979-696-4600 stata@stata.com
Stata license: 10-student 2-core lab perpetual
Serial number:
Licensed to: STATEANDO
Notes:
1. Unicode is supported; see help unicode_advice.
2. More than 2 billion observations are allowed; see help obs_advice.
3. Maximum number of variables is set to 5,000; see help set_maxvar.
4. New update available; type -update all-
Running C:\Users\User\Documents\profile.do ...
Ventana de resultados
Variables
Filter variables here
Name Label
There are no items to show.
Listado de Variables
Properties
Variables
Name
Label
Type
Format
Value label
Notes
Data
Frame default
Filename
Label
Notes
Variables 0
Observations 0
Size 0
Memory 64M
Sorted by
Command
Acá introducimos las líneas de código manualmente
C:\Users\User\Documents
CAP NUM INS

Sintaxis general

```
[by varlist:] comando [varlist] [=exp] [if exp] [in rango] [weight] [ , opc]
```

Sintaxis general

```
[by varlist:] comando [varlist] [=exp] [if exp] [in rango] [weight] [ , opc]
```



Sintaxis general

Una **expresión** (**exp**) es una combinación de valores, variables, operadores y funciones que Stata evalúa para devolver un resultado.

Ejemplos:

```
                  |     Expresiones   |

ingreso_real     = ingreso / ipc
edad_al_cuadrado = edad^2
precio_usd       = precio_peso * 1450
```

En la sintaxis, **[=exp]** indica la parte donde se escribe ese cálculo.

Sintaxis general

Con **if** podemos limitar la acción a las observaciones que cumplen con las expresiones lógicas.

Sintaxis general

Con **if** podemos limitar la acción a las observaciones que cumplen con las expresiones lógicas.

Operadores

Arithmetic		Logical		Relational (numeric and string)	
+	addition	&	and	>	greater than
-	subtraction	|	or	<	less than
*	multiplication	!	not	>=	> or equal
/	division	~	not	<=	< or equal
^	power			==	equal
-	negation			!=	not equal
+	string concatenation			~=	not equal

Sintaxis general

Con **if** podemos limitar la acción a las observaciones que cumplen con las expresiones lógicas.

Operadores

Arithmetic		Logical		Relational (numeric and string)	
+	addition	&	and	>	greater than
-	subtraction	|	or	<	less than
*	multiplication	!	not	>=	> or equal
/	division	~	not	<=	< or equal
^	power			==	equal
-	negation			!=	not equal
+	string concatenation			~=	not equal

Ejemplo:

```
if a == 2
if (a>1) & (a<5)
if (nombre == "Santiago") | (nombre == "San")
```

Sintaxis general

Con **in** podemos limitar la acción a un rango de observaciones. Ejemplo:

```
* De la obs 1 a la 10
in 1/10

* De la obs 5 a la última
in 5/l

* De la primera a la décima
in f/10
```

Sintaxis general

- Mensajes de error

```
. sysuse
invalid file specification
r(198);

. sysuse asd
file "asd.dta" not found
r(601);
```

- Help y documentación

```
[D] sysuse — Use shipped dataset
             (View complete PDF manual entry)


Syntax

    Use example dataset installed with Stata

        sysuse ["]filename["] [, clear]
```

Comentarios en el código

Los comentarios son documentación en el código mismo.

- **Claridad:** Explican qué hace cada sección y por qué.
- **Mantenibilidad:** Facilitan entender el código después de tiempo.
- **Colaboración:** Otros entienden rápidamente la lógica.
- **Debugging:** Ayudan a identificar y marcar problemas.
- **Decisiones metodológicas:** Justifican elecciones.

Formas de comentar en Stata

1. Asterisco (*) para comentarios en toda la línea
2. Barra doble (//) para comentarios en la misma línea
3. Bloques de comentarios con /* */

```
* Esto es un comentario que ocupa toda la línea (recomiendo usar este método).

// Esto también es un comentario. Stata ignora completamente esta línea

comando // también se puede comentar al final de una línea de código

/* Esto es un bloque de comentarios
   que puede ocupar varias líneas.
   En el medio NO puede haber código */
```

Contenido Clase 1

1. Recomendaciones

2. Interfaz y sintaxis general

3. Gestión básica de base de datos

4. Estadísticas descriptivas

5. Gráficos

6. Do-files

7. Trabajo Práctico 1

Importación de datos

Independientemente del formato del archivo (*.dta* , *.txt*, *.csv*, *etc*) es necesario decirle a Stata dónde está ese archivo (**ruta**).

Existen varias formas de hacerlo:

1. Usar la carpeta de trabajo actual (working directory).
2. Ruta absoluta: Especificar toda la ruta desde la raíz del sistema de archivos.
3. Ruta relativa: Especificar la ruta en relación a la carpeta de trabajo actual.

Importación de datos

1. Usar la carpeta de trabajo actual (working directory).

Utilizando el comando **cd** (change directory) podemos modificar la carpeta a donde va a apuntar Stata por defecto durante la sesión.

```
cd "C:/Users/usuario/Documents/datos"
```

Luego, para abrir un archivo en esa carpeta, solo es necesario usar el comando **use**:

```
use "archivo.dta"
```

Importación de datos

2. Ruta absoluta.

Directamente especificar la ruta al mismo tiempo que ejecutamos el comando para abrir el archivo

```
use "C:/Users/usuario/Documents/datosarchivo.dta"
```

Importación de datos

3. Ruta relativa.

Si conocemos la ruta relativa desde la carpeta de trabajo actual, podemos usarla para abrir el archivo.

Por ejemplo si la carpeta de trabajo actual es

```
C:/Users/usuario/Documents
```

y el archivo está en

```
C:/Users/usuario/Documents/datos/archivo.dta
```

podemos usar:

```
use "datos/archivo.dta"
```

Importación de datos

O si estamos en:

```
C:/Users/usuario/Documents/graficos
```

podemos usar:

```
use "../datos/archivo.dta"
```

Importación de datos

Debido a que Stata no permite trabajar con datasets simultaneos, para evitar errores al cargar nuevas bases debemos:

```
* Opción 1: usar el COMANDO "clear" previamente para borrar el dataset actual
clear
use "nombre del archivo.dta"

* Opción 2: usar la OPCIÓN "clear" del comando "use"
use "nombre del archivo.dta" , clear
```

Importación de datos

Para importar archivos de Excel (*.xls*, *.xlsx*) usamos: .

```
import excel "nombre del archivo.xlsx" , clear
```

para conservar los nombres de las variables (si es que estas se encuentran en la primer fila del archivo de Excel):

```
import excel "nombre del archivo.xlsx" , clear first
```

Importación de datos

Para importar archivos de texto con columnas delimitadas (*csv*, *txt*) usamos:

```
import delimited "nombre del archivo.csv" , clear
```

Importación de datos

Las opciones no son las mismas que para el **import excel**.

import_delimited_options	Description
delimiters("*chars*"[, collapse|asstring])	use *chars* as delimiters
varnames(#|nonames)	treat row # of data as variable names or the data do not have variable names
case(preserve|lower|upper)	preserve the case or read variable names as lowercase (the default) or uppercase
asfloat	import all floating-point data as **float**s
asdouble	import all floating-point data as **double**s
encoding(*encoding*)	specify the encoding of the text file being imported
stripquotes(yes|no|default)	remove or keep double quotes in data
bindquotes(loose|strict|nobind)	specify how to handle double quotes in data
maxquotedrows(# | unlimited)	number of rows of data allowed inside a quoted string when **bindquote(strict)** is specified
rowrange([*start*][:*end*])	row range of data to load
colrange([*start*][:*end*])	column range of data to load
parselocale(*locale*)	specify the locale to use for interpreting numbers in the text file being imported
decimalseparator(*character*)	character to use for the decimal separator when parsing numbers
groupseparator(*character*)	character to use for the grouping separator when parsing numbers
numericcols(*numlist*|_all)	force specified columns to be numeric
stringcols(*numlist*|_all)	force specified columns to be string
clear	replace data in memory
favorstrfixed	favor storing string variables as **str#** rather than **strL**

Exportación de datos

Para guardar un archivo en formato *.dta* debemos utilizar el comando **save**, teniendo en cuenta las mismas consideraciones respecto a la ubicación donde queremos crear el archivo.

```
save "ruta/nombre del archivo.dta"
```

Exportación de datos

Para guardar un archivo en formato *.dta* debemos utilizar el comando **save**, teniendo en cuenta las mismas consideraciones respecto a la ubicación donde queremos crear el archivo.

```
save "ruta/nombre del archivo.dta"
```

Si en esa ubicación ya existe un archivo *.dta* con el mismo nombre y queremos sobre-escribirlo:

```
save "ruta/nombre del archivo.dta" , replace
```

Exportación de datos

Para exportar archivos de Excel (*.xls*, *.xlsx*) y conservar los nombres de las variables:

```
export excel "ruta/nombre del archivo.xlsx" , replace firstrow(variable)
```

Exportación de datos

Para exportar archivos de Excel (*.xls*, *.xlsx*) y conservar los nombres de las variables:

```
export excel "ruta/nombre del archivo.xlsx" , replace firstrow(variable)
```

Para exportar archivos delimitados (*.csv*, *.txt*):

```
export delimited "ruta/nombre del archivo.csv" , replace
```

Inspección rápida de la base

El comando **`describe`** permite ver la estructura general del dataset cargado.

```
describe
```

Entre otras cosas, nos muestra:

- cuántas observaciones y variables tiene la base,
- el nombre y tipo de cada variable,
- las etiquetas de variables y de valores (si es que las tienen).

Inspección rápida de la base

Ejemplo:

```
sysuse lifeexp , clear
describe
```

```
Contains data from C:\Program Files\Stata19\ado\base/l/lifeexp.dta
 Observations:            68                  Life expectancy, 1998
    Variables:             6                  26 Mar 2024 09:40
                                              (_dta has notes)
-------------------------------------------------------------------------------------
Variable      Storage   Display    Value
    name         type    format    label      Variable label
-------------------------------------------------------------------------------------
region          byte    %16.0g     region     Region
country         str28   %28s                  Country
popgrowth       float   %9.0g               * Avg. annual  % growth
lexp            byte    %9.0g               * Life expectancy at birth
gnppc           float   %9.0g               * GNP per capita
safewater       byte    %9.0g               * Safe water
                                            * indicated variables have notes
-------------------------------------------------------------------------------------
Sorted by:
```

Tipos de variables

STATA puede almacenar variables de tipo:

- **Texto** (también llamadas cadena o string)
- **Numéricas**
- **Fechas**

Tipos de variables numéricas

Nombre	Tipo de número	Límite inferior	Límite superior
Byte	Enteros	-127	100
Integer	Enteros	-32.767	32.740
Long	Enteros	-2.147.486.947	2.147.486.947
Float	Con decimales	Precisión de hasta 7 decimales	
Double	Con decimales	Precisión de hasta 16 decimales	

Tipos de variables

Valores faltantes (missing values)

- **Texto:** missing = **“ ”** (cadena vacía)
- **Numéricas:** missing = **.**
  - Cualquier operación aritmética que se haga con ellos arrojará otro valor missing
  - En cuanto al orden, el missing es considerado por STATA como el valor más alto de todos

Creación de variables

El comando **generate** permite crear nuevas variables (columnas) en la base de datos.

La sintaxis del comando es:

```
generate newvar = exp [if] [in]
```

```
  * Ejemplos
generate variable1 = 3
generate variable2 = _n
generate variable3 = variable1 + variable2 if variable2 < 10
```

Eliminación de variables

Para borrar una o más variables podemos utilizar el comando **`drop`**:

```
drop variable1 variable2
```

Si, por ejemplo, quisiéramos borrar todas las variables que empiezan con la letra v podemos usar el asterisco a modo de caracter "comodín".

```
drop v*
```

Eliminación de variables

Otra forma es mantener solo las variables con las que queremos trabajar, descartando el resto:

```
keep variable3 variable4
```

De igual manera, si quisiéramos quedarnos solo con las variables que terminan con 3 o con 4

```
keep *3 *4
```

Eliminación de observaciones

Los comandos **drop** y **keep** también permiten borrar o mantener observaciones

```
drop  if año > 1992
```

Análogamente,:

```
keep if año <= 1992
```

Nombres de variables

- NO pueden contener espacios ni comenzar con números.
- Pueden tener hasta 32 caracteres.
- Por convención, se sugiere usar letras minúsculas, sin acentos.

Si queremos renombrar la *variable1* para que pase a llamarse *id*:

```
rename variable1 id
```

Nombres de variables

- NO pueden contener espacios ni comenzar con números.
- Pueden tener hasta 32 caracteres.
- Por convención, se sugiere usar letras minúsculas, sin acentos.

Si queremos renombrar la *variable1* para que pase a llamarse *id*:

```
rename variable1 id
```

Si queremos cambiar el prefijo de todas las variables que comienzan con *var* para que pasen a llamarse *v_1*, *v_2*, *etc*:

```
rename var* v_*
```

Modificación de datos

```
replace id = 1
```

La columna correspondiente a la variable *id* pasa a contener 1 en todas las observaciones (filas).

Por supuesto que estos cambios pueden realizarse de manera condicional utilizando tanto **`if`** como **`in`**.

Modificación de datos

Práctica habitual:

```
generate menor = .
replace  menor = 1 if edad < 18
replace  menor = 0 if edad >= 18 & edad != .
```

Alternativamente:

```
generate menor = (edad < 18)
```

Pregunta

¿ Es lo mismo ?

Modificación de datos

```
generate menor = (edad < 18)
```

La condición lógica **(edad $<$ 18)**

- toma el valor 1 (true) cuándo la condición se cumple.
- toma el valor 0 (false) cuándo no se cumple.

Pregunta

¿ Qué pasa si edad tiene missing ?

Etiquetas de variables

Si queremos poner una etiqueta que diga **"Menor de edad"** a la variable *menor* para tener siempre presente qué significa esa variable:

```
1   label variable menor "Menor de edad"
```

Etiquetas de valores

Si, en cambio, lo que queremos es etiquetar los valores que toma la variable *menor* para saber qué significa que valga 0 ó 1:

```
label define menor_lbl 0 "No" 1 "Sí"
label values menor menor_lbl
```

Ordenamiento

```
sort varlist [in] [ , stable]
```

- Organiza las observaciones en orden **ascendente** basado en los valores de las variables que especifiquemos.
- Si especificamos varias variables, el ordenamiento se hace primero por la primera, luego por la segunda dentro de los grupos formados por la primera, y así sucesivamente.
- Missing numéricos se colocan en **último lugar**.
- Missing de strings se colocan **al principio**.
- Stable permite que las observaciones con los mismos valores de las variables en varlist mantengan el mismo orden relativo que tenían previamente.

Ordenamiento

Pregunta

¿ Y si queremos ordenar de mayor a menor ?

Ordenamiento

Pregunta

¿ Y si queremos ordenar de mayor a menor ?

```
gsort [+|-] varname [[+|-] varname ...] [ , generate(newvar) mfirst]
```

- Permite ordenar tanto de manera **ascendente** como **descendente** usando el signo más ó menos antes del nombre de la variable, respectivamente.
- OJO que las opciones son distintas a las del comando **sort**.
- La opción **mfirst** permite colocar los missing numéricos al **principio** del ordenamiento.

Contenido Clase 1

1. Recomendaciones

2. Interfaz y sintaxis general

3. Gestión básica de base de datos

4. Estadísticas descriptivas

5. Gráficos

6. Do-files

7. Trabajo Práctico 1

Estadísticas básicas

- Existen muchos comandos para obtener estadísticas descriptivas.
- En esta clase veremos los más básicos y comunes: **`summarize`**, **`tabulate`**, **`tabstat`** y **`table`**.
- Tener en claro las particularidades y opciones de cada uno es clave para **evitar errores** en los análisis y simplificar la **exportación de resultados**.

Estadísticas básicas

Utilizaremos la base de datos de ejemplo llamada auto.dta, que se instala automáticamente con Stata.

Para acceder a ella de manera directa:

```
sysuse auto , clear
```

Estadísticas básicas

El comando **summarize** permite obtener estadísticas descriptivas de una o más variables.

```
* Para todas las variables del dataset
summarize

* Para algunas variables
summarize price mpg weight

* Obviamente también podemos restringir observaciones con if o in
summarize length if price > 6000
```

Estadísticas básicas

```
* Puede abreviarse como "sum"
sum price
```

```
    Variable |        Obs        Mean    Std. dev.       Min        Max
-------------+---------------------------------------------------------
       price |         74    6165.257    2949.496       3291      15906
```

Estadísticas básicas

```
* Podemos agregar la opción detail para más estadísticas
sum price , detail
```

```
                          Price
-------------------------------------------------------------
      Percentiles      Smallest
 1%         3291           3291
 5%         3748           3299
10%         3895           3667       Obs                  74
25%         4195           3748       Sum of wgt.          74

50%       5006.5                      Mean           6165.257
                        Largest       Std. dev.      2949.496
75%         6342          13466
90%        11385          13594       Variance        8699526
95%        13466          14500       Skewness       1.653434
99%        15906          15906       Kurtosis       4.819188
```

Estadísticas básicas

Para tabular la **cantidad de observaciones** según los distintos valores que puede tomar una variable (también llamadas frecuencias), utilizamos el comando **`tabulate`**.

```
tabulate rep78
```

```
     Repair |
record 1978 |      Freq.     Percent        Cum.
------------+-----------------------------------
          1 |          2        2.90        2.90
          2 |          8       11.59       14.49
          3 |         30       43.48       57.97
          4 |         18       26.09       84.06
          5 |         11       15.94      100.00
------------+-----------------------------------
      Total |         69      100.00
```

Estadísticas básicas

Agregando la opción missing incluimos la cantidad de obs con valores faltantes

```
tabulate rep78 , missing
```

```
     Repair |
record 1978 |      Freq.     Percent        Cum.
------------+-----------------------------------
          1 |          2        2.70        2.70
          2 |          8       10.81       13.51
          3 |         30       40.54       54.05
          4 |         18       24.32       78.38
          5 |         11       14.86       93.24
          . |          5        6.76      100.00
------------+-----------------------------------
      Total |         74      100.00
```

Estadísticas básicas

Otra opción sumamente útil que admite es **summarize**

```
tabulate rep78 , summarize(price)
```

```
     Repair |          Summary of Price
record 1978 |        Mean   Std. dev.       Freq.
------------+------------------------------------
          1 |     4,564.5   522.55191           2
          2 |   5,967.625   3,579.357           8
          3 |   6,429.233    3,525.14          30
          4 |     6,071.5   1,709.608          18
          5 |       5,913   2,615.763          11
------------+------------------------------------
      Total |   6,146.043    2,912.44          69
```

Estadísticas básicas

También puede usarse para cruce de dos variables

```
tabulate rep78 foreign
```

```
    Repair |
    record |      Car origin
      1978 |  Domestic    Foreign |     Total
-----------+----------------------+----------
         1 |         2          0 |         2
         2 |         8          0 |         8
         3 |        27          3 |        30
         4 |         9          9 |        18
         5 |         2          9 |        11
-----------+----------------------+----------
     Total |        48         21 |        69
```

Estadísticas básicas

tabstat sirve para obtener estadísticos específicos de una o más variables

```
tabstat price , statistics(mean)
```

```
    Variable |      Mean
-------------+----------
       price |  6165.257
------------------------
```

```
tabstat price length weight , s(mean min max)
```

```
   Stats |     price    length    weight
---------+------------------------------
    Mean |  6165.257  187.9324  3019.459
     Min |      3291       142      1760
     Max |     15906       233      4840
----------------------------------------
```

Estadísticas básicas

`table` tal vez sea el comando más versátil para obtener estadísticas descriptivas. Pero también es el más complejo y con más opciones, por lo que es necesario conocerlo bien para evitar errores.

```
Syntax

    Basic syntax for a one-way table

        table rowvar

        table () colvar

    Basic syntax for a two-way table

        table rowvar colvar

    Basic syntax for an n-way table

        table rowvars colvar

        table rowvar (colvars)

        table (rowvars) (colvars)

    Basic syntax for multiple n-way tables

        table (rowvars) (colvars) (tabvars)

    Full syntax

        table (rowspec) (colspec) [(tabspec)] [if] [in] [weight] [, options]
```

Estadísticas básicas

Obviamente que se pueden hacer tabulados (incluso con más de dos variables):

```
table rep78 foreign
```

```
----------------------------------------------
                   |          Car origin
                   |  Domestic   Foreign   Total
-------------------+--------------------------
Repair record 1978 |
  1                |         2                 2
  2                |         8                 8
  3                |        27         3      30
  4                |         9         9      18
  5                |         2         9      11
  Total            |        48        21      69
----------------------------------------------
```

Estadísticas básicas

Incluir estadísticos:

```
table rep78 foreign , stat(min price)
```

```
----------------------------------------------
                   |          Car origin
                   |  Domestic   Foreign   Total
-------------------+--------------------------
Repair record 1978 |
  1                |      4195              4195
  2                |      3667              3667
  3                |      3291      3895    3291
  4                |      3829      3995    3829
  5                |      3984      3748    3748
  Total            |      3291      3748    3291
----------------------------------------------
```

Estadísticas básicas

Y hasta hacer regresiones:

```
table (colname result) (foreign), ///
    command(_r_b _r_se: regress price weight ) ///
    nformat(%6.2f) ///
    sformat("(%s)" _r_se) ///
    style(table-reg3) ///
    notot
```

```
---------------------------------------
              |        Car origin
              |   Domestic     Foreign
--------------+------------------------
Weight (lbs.) |     2.99        5.36
              |    (0.47)      (0.63)
              |
    Intercept |   -3861.72    -6033.32
              |  (1579.52)   (1480.90)
---------------------------------------
```

Estadísticas básicas

A partir de la versión 17 sufrió muchas actualizaciones. Pero para estadísticas descriptivas básicas la sintaxis es bastante sencilla:

```
                                  estadístico
                                      |
table   var1       var2     ,   stat(xxx var4)
         |          |          |____________|
      división   división        contenido
       filas     columnas       de las celdas
```

Estadísticas básicas

Y entonces ... Cuál usar ?

Estadísticas básicas

Y entonces ... Cuál usar ?

Dependerá de:

- Qué estadísticas quiero obtener.
- Cómo quiero exportar los resultados (si es que quiero exportarlos).
- La versión de Stata que tengo disponible.
- Gusto/preferencia personal.

Contenido Clase 1

1. Recomendaciones

2. Interfaz y sintaxis general

3. Gestión básica de base de datos

4. Estadísticas descriptivas

5. Gráficos

6. Do-files

7. Trabajo Práctico 1

Gráficos

Familias de gráficos

El comando que nos permite realizar gráficos es **`graph`**. Siempre se utiliza acompañado de un segundo comando que indica **la familia** a la cual pertenece.

Gráficos

Familias de gráficos

El comando que nos permite realizar gráficos es **graph**. Siempre se utiliza acompañado de un segundo comando que indica **la familia** a la cual pertenece.

```
graph twoway /// para relacionar dos variables numéricas
graph bar    /// para gráficos de barras
graph box    /// para gráficos de caja
graph pie    /// para gráficos de torta
```

Existen muchos más (ver **`help graph`**)

Comandos auxiliares para gráficos

Algunos comandos que pueden utilizarse con cualquier familia de gráficos:

```
graph save       /// guarda gráficos en formato .gph
graph use        /// abre gráficos guardados en formato .gph
graph display    /// abre gráficos guardados temporalmente
graph combine    /// combina múltiples gráficos
graph export     /// exporta gráficos a formatos como PNG, PDF, etc.
```

Gráficos

Familia twoway

La sintaxis para la familia de gráficos twoway:

```
[graph] twoway plot [if] [in] [ , twoway options]
```

En donde la sintaxis de plot es:

```
[(] plot_type yvar xvar [ , options [)]
```

Entre algunos de los plottypes se encuentran:

```
scatter   /// gráfico de dispersión
line      /// gráfico de líneas
connected /// gráfico de líneas conectadas
```

Opciones

Si bien cada familia de gráficos (e incluso cada plot) tiene sus propias opciones, existen algunas que son comunes a todas

```
title()      /// título del gráfico
subtitle()   /// subtítulo del gráfico
scheme()     /// esquema de colores y estilo del gráfico
xlabel()     /// etiquetas del eje x
ylabel()     /// etiquetas del eje y
```

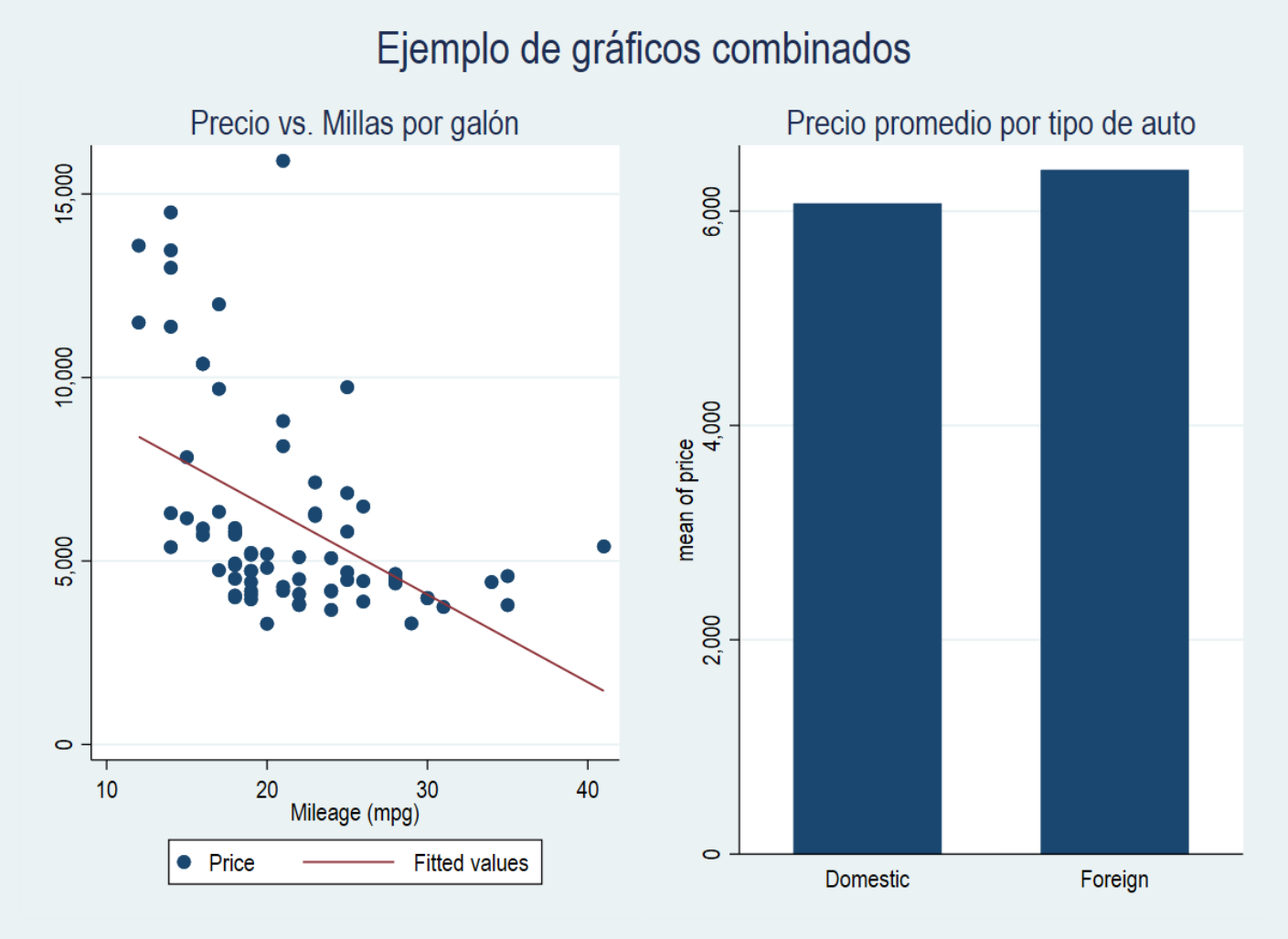
Gráficos

Ejemplo:

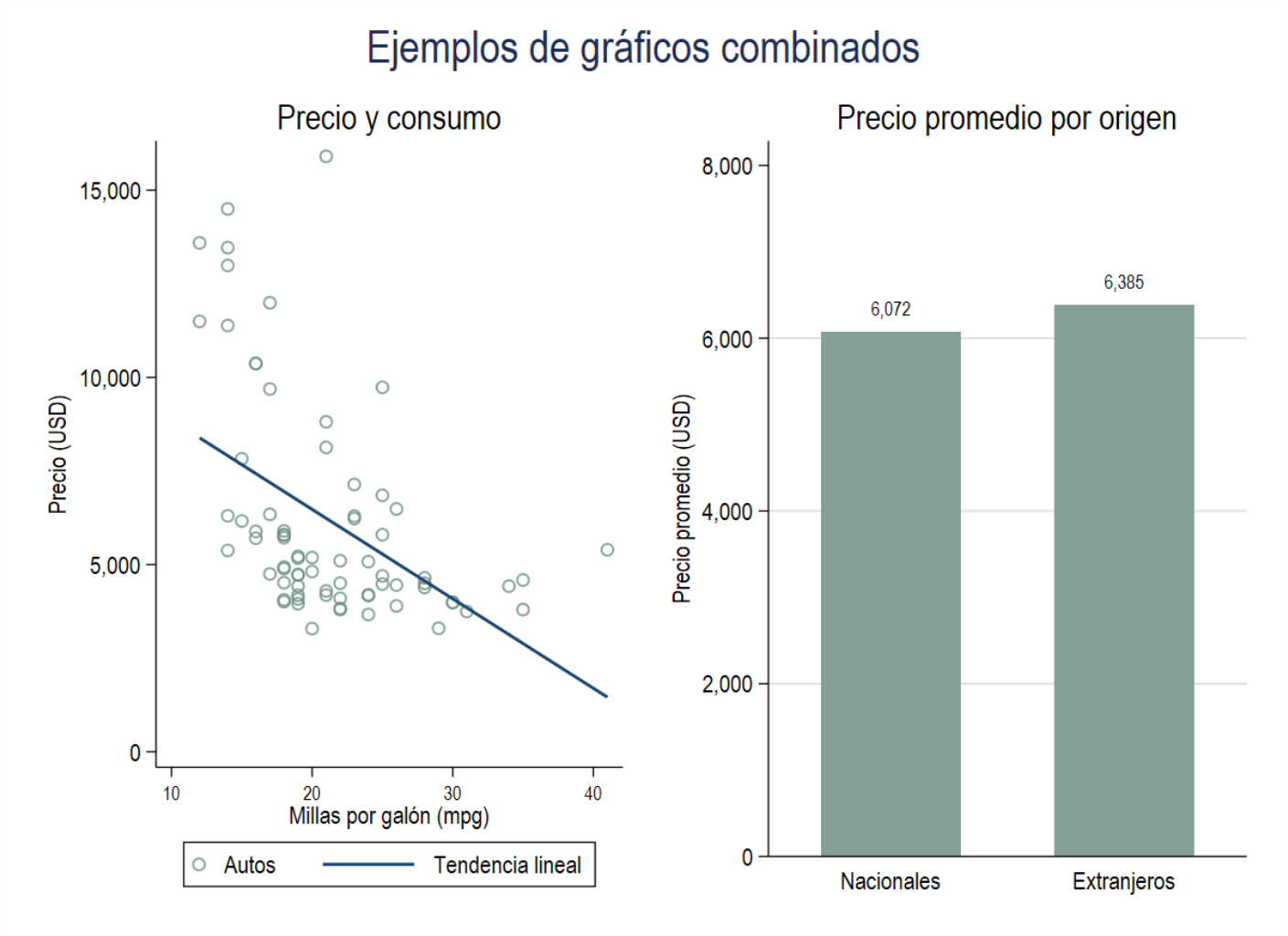
```
* Cargar la base de datos
sysuse auto, clear

* Gráfico de dispersión
twoway (scatter price mpg) (lfit price mpg), name(g1, replace) title("Precio
      vs. Millas por galón")

* Gráfico de barras
graph bar (mean) price, name(g2, replace) over(foreign) title("Precio
     promedio por tipo de auto")

* Combinar gráficos
graph combine g1 g2, title("Ejemplo de gráficos combinados")
```

Gráficos

Resultado:



Resultado agregando opciones para mejorar el estilo:



Gráficos

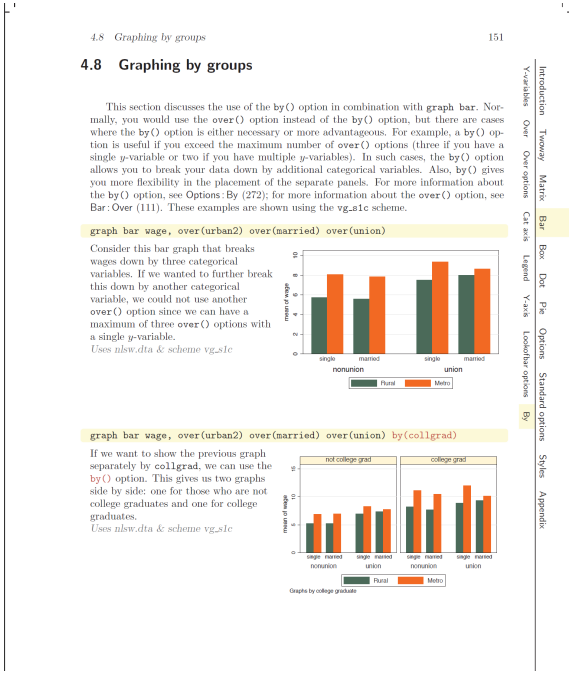
Obviamente el código se complica un poco más:

```
* Gráfico de dispersión
twoway ///
    (scatter price mpg , ///
        msymbol(circle_hollow) ///
        msize(medium) ///
        mcolor(teal%80)) ///
    (lfit price mpg , ///
        lcolor(navy) ///
        lwidth(medthick)) , ///
    name(g1 , replace) ///
    title("Precio y consumo") ///
    xtitle("Millas por galón (mpg)") ///
    ytitle("Precio (USD)") ///
    legend(order(1 "Autos" 2 "Tendencia lineal") pos(6) rows(1)) ///
    graphregion(color(white)) ///
    plotregion(color(white)) ///
    ylabel( , angle(horizontal) format(%9.0fc)) ///
    xlabel( , labsize(small)) ///
    scheme(s1color)

* Gráfico de barras
* ...
```

Gráficos

Una buena referencia para realizar gráficos y ver las posibles variantes y opciones disponibles para cada tipo es:

- **Mitchell, M. N. (2008)**: *A visual guide to Stata graphics* Stata Press books.



4.8 Graphing by groups 151

4.8 Graphing by groups

This section discusses the use of the `by()` option in combination with `graph bar`. Normally, you would use the `over()` option instead of the `by()` option, but there are cases where the `by()` option is either necessary or more advantageous. For example, a `by()` option is useful if you exceed the maximum number of `over()` options (three if you have a single y-variable or two if you have multiple y-variables). In such cases, the `by()` option allows you to break your data down by additional categorical variables. Also, `by()` gives you more flexibility in the placement of the separate panels. For more information about the `by()` option, see Options : By (272); for more information about the `over()` option, see Bar : Over (111). These examples are shown using the `vg_s1c` scheme.

```
graph bar wage, over(urban2) over(married) over(union)
```

Consider this bar graph that breaks wages down by three categorical variables. If we wanted to further break this down by another categorical variable, we could not use another `over()` option since we can have a maximum of three `over()` options with a single y-variable.
Uses nlsw.dta & scheme vg_s1c


```
graph bar wage, over(urban2) over(married) over(union) by(collgrad)
```

If we want to show the previous graph separately by `collgrad`, we can use the `by()` option. This gives us two graphs side by side: one for those who are not college graduates and one for college graduates.
Uses nlsw.dta & scheme vg_s1c

Introduction Twoway Matrix Bar Box Dot Pie Options Standard options Styles Appendix
Y-variables Over Over options Cat axis Legend Y-axis Lookfor options By

Contenido - Clase 2

1. Recomendaciones

2. Interfaz y sintaxis general

3. Gestión básica de base de datos

4. Estadísticas descriptivas

5. Gráficos

6. Do-files

7. Trabajo Práctico 1

Do-files

Los comandos pueden introducirse:

1. De forma inmediata a través de la ventana *comandos*.
2. A través de archivos de texto que Stata interpreta y ejecuta de manera secuencial (archivos .do).

Do-files

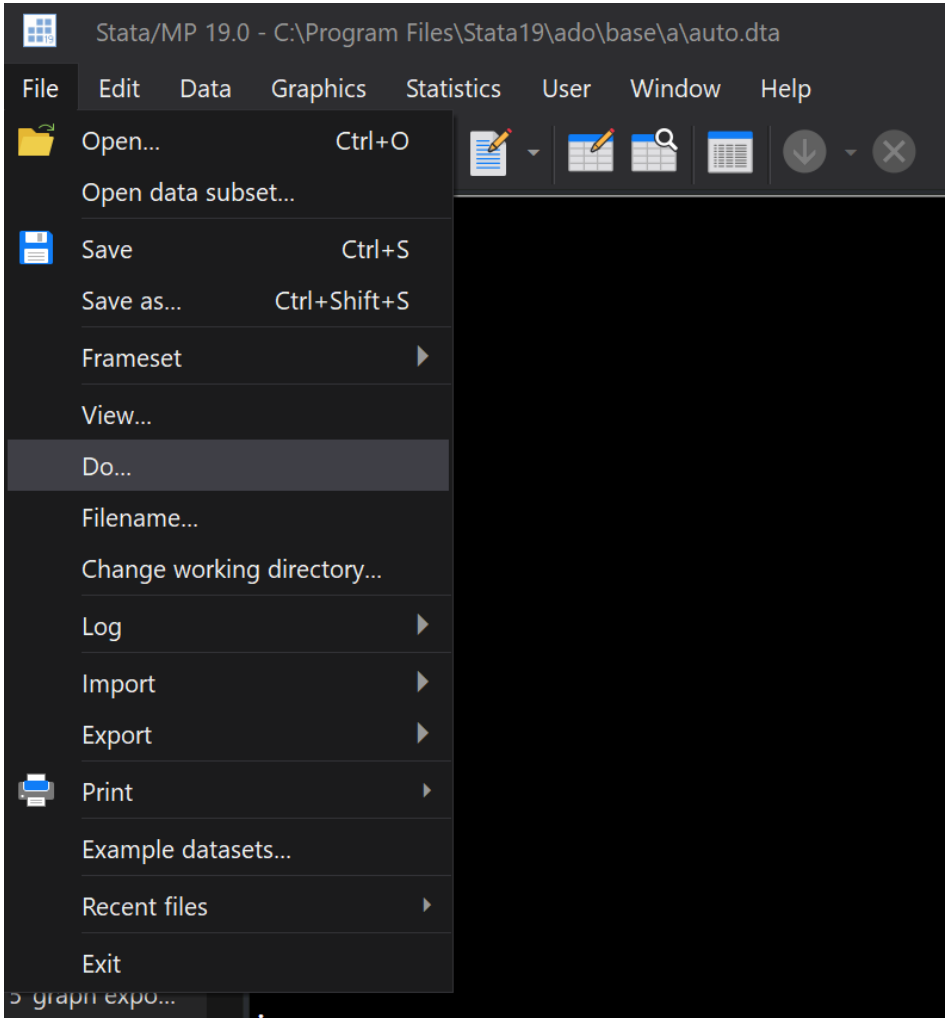
Los comandos pueden introducirse:

1. De forma inmediata a través de la ventana *comandos*.
2. A través de archivos de texto que Stata interpreta y ejecuta de manera secuencial (archivos .do).

```
*// dofile que resuelve el Ejercicio

clear all

* Ruta
cd "C:\MiCarpeta\ejercicio"

* Punto 1
use "ejercicio.dta" , clear

* Punto 2
rename price precio_dolar

* Punto 3
gen precio_peso = precio_dolar * 1500

* Punto 4
save "ejercicio_modificada.dta" , replace
```

Do-files

Ejecución desde la interfaz



Do-files

Ejecución con el comando do

```
do "ruta del dofile \ nombre del dofile"
```

Do-files

Ejecución con el comando do

```
do "ruta del dofile \ nombre del dofile"
```

En ambos casos, si Stata detecta un error, detendrá la ejecución del dofile y no ejecutará las líneas siguientes.

Contenido Clase 1

1. Recomendaciones

2. Interfaz y sintaxis general

3. Gestión básica de base de datos

4. Estadísticas descriptivas

5. Gráficos

6. Do-files

7. Trabajo Práctico 1

Trabajo Práctico 1

- **Objetivo:** Practicar los contenidos vistos en esta clase y familiarizarse con la creación de do-files.
- **Tarea:** Crear un do-file que realice las siguientes tareas usando la base de datos `auto.dta` incluida en Stata.
- **Entrega:** Debe enviarse al mail santiagocerutti1@gmail.com antes de la próxima clase (14/7).
- **Importante:** El código debe poder ejecutarse por completo desde cualquier PC, incluso si ya fue ejecutado previamente. Se valorará la prolijidad y el uso de comentarios adecuados.

TP 1 - Parte 1: Carga y exploración

1. Cargar la base de datos `auto.dta`.
2. Explorar la base: ¿cuántas observaciones y variables tiene?
3. Obtener un resumen estadístico general de todas las **variables numéricas**.

TP 1 - Parte 2: Variables y etiquetas

4. Renombrar la variable **`weight`** a **`peso`** y Asignarle la etiqueta *"Peso (libras)"*.
5. Crear una variable **`precio_miles`** que contenga el precio dividido por 1000. Asignarle la etiqueta *"Precio (miles de USD)"*.
6. Crear una variable **`pesado`** que valga 1 si el peso es mayor a 3000 libras y 0 en caso contrario. Asignarle la etiqueta *"Vehículo pesado"*.
7. Definir etiquetas de valores para **`pesado`**: 0 $\rightarrow$ "Liviano", 1 $\rightarrow$ "Pesado". Asignarlas a la variable.

TP 1 - Parte 3: Limpieza y ordenamiento

8. Eliminar las variables **`headroom`** y **`trunk`**.
9. Eliminar las observaciones cuya variable **`rep78`** sea missing.
10. Ordenar la base por **`precio`** de mayor a menor.

TP 1 - Parte 4: Estadísticas descriptivas

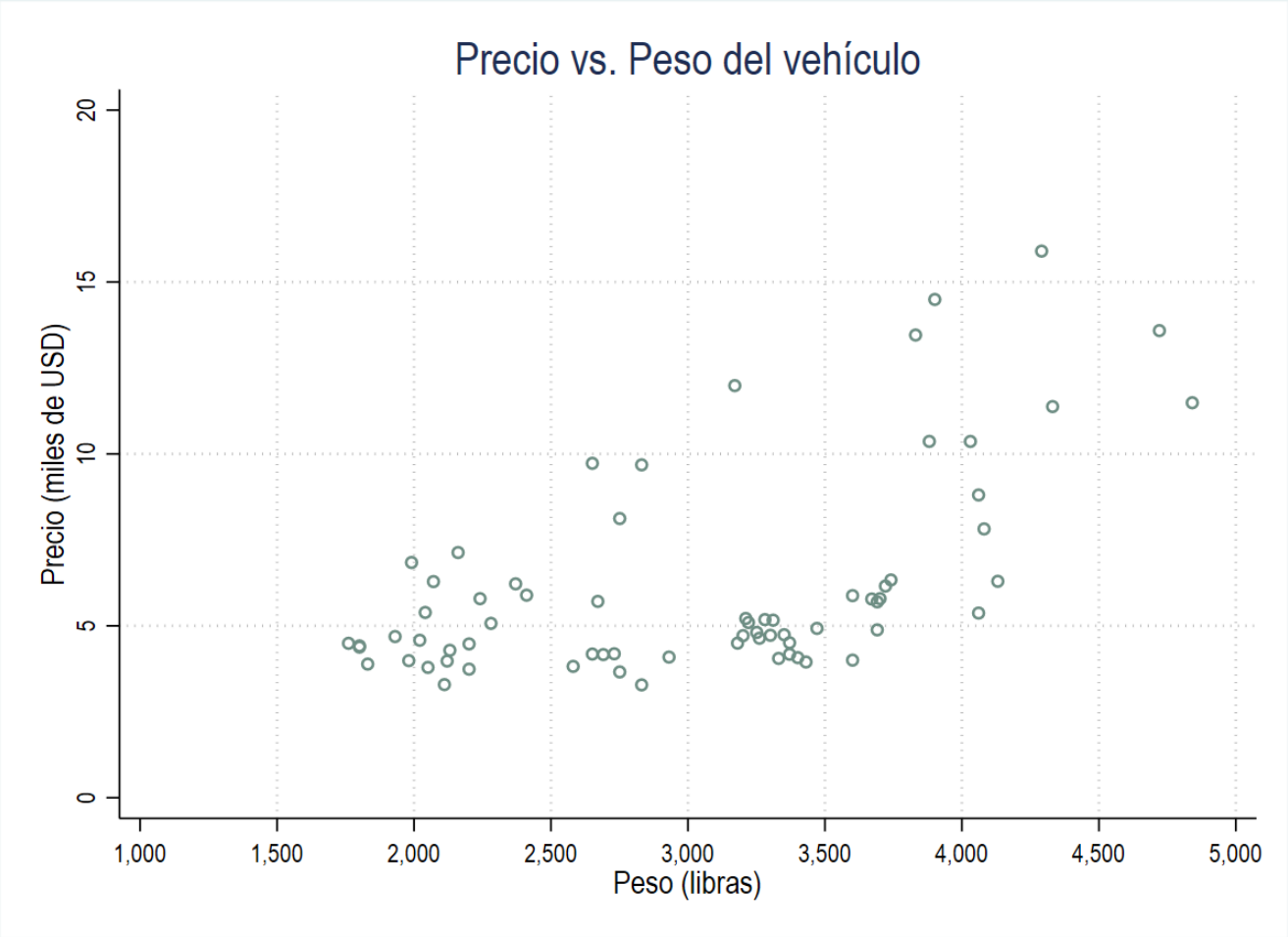
11. Crear una tabla que muestre, entre los autos con precio mayor a 4000, la cantidad y el porcentaje de autos nacionales y extranjeros según sean livianos o pesados.

12. Crear una tabla que muestre, según el origen del auto, el promedio y el desvío estándar de **`precio_miles`** y **`peso`**. Intentar replicar exactamente el formato para que se vea así:

```
----------------------------------------------------------------------------------------
           |                   Mean                              Standard deviation
           |  Precio (miles de USD)   Peso (libras)   Precio (miles de USD)   Peso (libras)
-----------+----------------------------------------------------------------------------
Car origin |
  Domestic |                   6.18        3,368.33                     3.2           688.0
  Foreign  |                   6.07        2,263.33                     2.2           364.7
  Total    |                   6.15        3,032.03                     2.9           792.9
----------------------------------------------------------------------------------------
```

TP 1 - Parte 5: Gráfico

13. Replicar el gráfico de dispersión (**`scatter`**) de **`precio_miles`** contra **`peso`** respetando el estilo lo máximo posible.



TP 1 - Parte 6: Exportación

14. Exportar el gráfico de dispersión a un archivo PNG con el nombre `scatter_ejercicio1.png`.
15. Guardar la base de datos modificada con el nombre `auto_modificada.dta`.